

מערכת ספירת ז'אנגלינג בזמן אמת

ניתוח מקיף של המערכת, האלגוריתמים, ושלבי הכיול

Juggling Live --- קבוצה 18

27 במאי 2026

מסמך זה מהווה ניתוח טכני מקיף של פרויקט Juggling Live, מערכת מבוססת ראייה ממוחשבת לזיהוי וספירת בעיטות ז'אנגלינג בזמן אמת באמצעות שתי מצלמות (עליונה וצדדית). המסמך סוקר את ארכיטקטורת המערכת, צינור עיבוד התמונה, האלגוריתמים המרכזיים בהם נעשה שימוש (MOG2, HSV, Hough Circles), שלבי הכיול השונים, לוגיקת המשחק וההיגיון מאחורי כל החלטה תכנונית. (Kalman Filter, Homography)

תוכן העניינים

2	1	מבוא ומטרת הפרויקט
2	1.1	אתגרים מרכזיים
2	1.2	גישת הפתרון
2	2	ארכיטקטורת המערכת
2	2.1	מבנה מודולרי של הפרויקט
3	2.2	זרימת מידע ראשית
3	3	צינור עיבוד התמונה
3	3.1	שלב 1: רכישה ועיבוד מקדים
3	3.2	שלב 2: מסיכת תנועה -- MOG2
3	3.3	שלב 3: מסיכת צבע -- HSV Thresholding
3	3.4	שלב 4: מיזוג היברידי
4	3.5	שלב 5: פעולות מורפולוגיות
4	4	אלגוריתמי הזיהוי
4	4.1	זיהוי קפדני -- Hough Circle Transform
4	4.2	זיהוי גמיש -- Contour Analysis
4	4.3	החלקה ותחזית -- Kalman Filter
5	5	זיהוי רצפה באמצעות הומוגרפיה
5	5.1	הרעיון
5	5.2	מתמטית
5	5.3	מימוש

6	שלבי הכיול	6
6	שלב 0: כיול HSV חיצוני	6.1
6	שלב 1: כיול רקע -- מקש B	6.2
6	שלב 2: כיול רדיוס -- מקש S	6.3
6	שלב 3: כיול רצפה ב-12 נקודות -- מקש F	6.4
7	שלב 4: כיול נגיעת ראש -- מקש H, אופציונלי	6.5
7	שלב 5: כוונון קו רצפה ידני	6.6
7	לוגיקת זיהוי בעיטה ונפילה	7
7	זיהוי בעיטה -- Velocity Flip	7.1
7	בעיטה ראשונה -- Flick Up	7.2
8	זיהוי נפילה לרצפה -- שלוש שיטות במקביל	7.3
8	7.3.1 שיטה 1: הומוגרפיה (מועדפת)	
8	7.3.2 שיטה 2: קו רצפה ליניארי (fallback)	
8	7.3.3 שיטה 3: ניתוח רטרואקטיבי של תזמון פגיעות	
8	זיהוי נגיעת ראש -- Header	7.4
8	מנגנונים נוספים	8
8	טיפול בחסימות -- Lost Frame Handling	8.1
9	שער רדיוס -- Radius Gate	8.2
9	משחק רב משתתפים	8.3
9	משוב קולי	8.4
9	תצוגה וויזואליזציה	9
9	Floor Overlay	9.1
9	הודעות תוצאה	9.2
10	קבצי קונפיגורציה	10
10	הערכה -- Evaluation	11
10	סיכום ומסקנות	12

1 מבוא ומטרת הפרויקט

הפרויקט נועד לפתח מערכת אוטומטית לזיהוי וספירת בעיטות זיאנגלינג של כדור רגל בזמן אמת, באמצעות ראייה ממוחשבת בלבד וללא חיישנים פיזיים מותקנים על הכדור או על השחקן.

1.1 אתגרים מרכזיים

- **שונוות סביבתית** --- תנאי תאורה משתנים, רקע דינמי וצבעי כדור משתנים.
- **חסימות (Occlusions)** --- הכדור עלול להיעלם זמנית מאחורי גוף השחקן.
- **הפרדה בין בעיטה לנפילה לרצפה** --- שני האירועים יוצרים שינוי כיוון של הכדור כלפי מעלה, ויש להבחין ביניהם במהירות גבוהה.
- **זיהוי נגיעת ראש (header)** --- אירוע שמתבצע בציר Z (לכיוון או מהמצלמה העליונה) ולא בציר Y.
- **דיוק זמן אמת** --- כל הפעולות חייבות לרוץ במסגרת קצב הפריימים של המצלמות.

1.2 גישת הפתרון

המערכת משלבת **שתי מצלמות (top + side)** שמספקות שתי תצוגות בלתי תלויות של אותה זירת משחק. שילוב המידע משתי המצלמות מאפשר:

1. הצלבת זיהויי כדור והפחתת False Positives.
2. שימוש בהומוגרפיה (Homography) לחישוב מיקום אמיתי של הכדור במישור הרצפה.
3. זיהוי נגיעות ראש על ידי ניתוח השינוי ברדיוס הכדור במצלמה העליונה.

2 ארכיטקטורת המערכת

2.1 מבנה מודולרי של הפרויקט

המערכת מאורגנת כצינור (pipeline) של מודולים בעלי אחריות בודדת, על פי עיקרון Single Responsibility:

- **main.py** --- לולאה ראשית, ניהול קלט מקלדת, רינדור UI, תזמור הכיולים.
- **camera_manager.py** --- ניהול מצלמות בתהליכון (thread) נפרד.
- **ball_processor.py** --- צינור זיהוי הכדור לכל מצלמה בנפרד.
- **juggling_logic.py** --- לוגיקת המשחק, ספירת בעיטות, זיהוי נפילה.
- **floor_finding.py** --- מודול הומוגרפיה לזיהוי מגע ברצפה.
- **config.py + config_utils.py** --- ניהול קונפיגורציה ושמירת הגדרות.
- **Calibration Helper script.py** --- כלי חיצוני לכיול HSV.
- **evaluate_segmentation.py** --- הערכת איכות סגמנטציה מול ground truth.

2.2 זרימת מידע ראשית

1. DualCameraManager מספק שני פריימים סינכרוניים (side, top).
2. כל פריימים מועבר ל-BallProcessor מתאים, שמחזיר (x, y, r) של הכדור.
3. שני זיהויי הכדור מועברים ל-JugglingCounter.update(), שמעדכן את מצב המשחק.
4. תוצאות הציור מתבצעות בלולאה הראשית ומוצגות בשני חלונות OpenCV.

הערה ארכיטקטונית: המצלמה הצדדית (side) משמשת כמאסטר --- היא קובעת אם המערכת זיהתה כדור בכלל. המצלמה העליונה (top) משמשת כסלייב --- היא משתמשת במצב relaxed (חיפוש קונטור) רק כאשר המצלמה הצדדית הצליחה לזהות את הכדור. זה מפחית רעש ושומר על דיוק גבוה.

3 צינור עיבוד התמונה

הצינור פועל בכל פריימים בנפרד ומורכב מהשלבים הבאים:

3.1 שלב 1: רכישה ועיבוד מקדים

- שינוי גודל אחיד ל- 360×640 פיקסלים (INTER_AREA).
- חיתוך ל-ROI (Region Of Interest) להפחתת רעש.
- טשטוש גאוסיאני (GaussianBlur 11×11) להחלקת הפיקסלים.

3.2 שלב 2: מסיכת תנועה -- MOG2

המערכת משתמשת באלגוריתם **Mixture of Gaussians 2** של OpenCV לזיהוי פיקסלים בתנועה. קוד אתחול המסנן ב-ball_processor.py:

```
self.fgbg = cv2.createBackgroundSubtractorMOG2(  
    history=CONFIG["background"].get("mog2_history", 500),  
    varThreshold=CONFIG["background"].get("mog2_threshold", 25),  
    detectShadows=False  
)
```

עיקרון פעולה: כל פיקסל מתואר על-ידי תערובת של גאוסיאנים שמייצגים את התפלגות הצבעים שלו לאורך זמן. פיקסלים שחורגים מההתפלגות הנלמדת מסומנים כתנועה (foreground). **היתרון:** עמידות לרעש ושינויי תאורה איטיים.

3.3 שלב 3: מסיכת צבע -- HSV Thresholding

```
hsv = cv2.cvtColor(blurred, cv2.COLOR_BGR2HSV)  
color_mask = cv2.inRange(hsv, self.lower_hsv, self.upper_hsv)
```

המעבר ממרחב BGR ל-HSV מאפשר לבדוד צבע לפי גוון (Hue) ללא תלות בעוצמת התאורה. ערכי החסם (lower_hsv, upper_hsv) נטענים מקובץ ball_config.json, אחד לכל מצלמה.

3.4 שלב 4: מיזוג היברידי

שתי המסיכות מאוחדות ב-AND לוגי:

```

if self.mog_bypass_frames > 0:
    final_mask = color_mask # color-only mode (calibration)
    self.mog_bypass_frames -= 1
else:
    final_mask = cv2.bitwise_and(color_mask, color_mask, mask=fgmask)

```

זה מבטיח שרק אזורים שגם זזים וגם בצבע הנכון ייחשבו כמועמדים לכדור.

3.5 שלב 5: פעולות מורפולוגיות

- Opening (3×3 kernel) --- מסיר רעש קטן.
- Closing (11×11 kernel) --- סוגר חורים בתוך הכדור.

4 אלגוריתמי זיהוי

4.1 זיהוי קפדני -- Hough Circle Transform

שיטה זו (מצב strict) מחפשת מעגלים מושלמים גיאומטרית:

```

circles = cv2.HoughCircles(
    mask_blurred_for_hough,
    cv2.HOUGH_GRADIENT,
    dp=1.2, minDist=100,
    param1=50, param2=20,
    minRadius=self.min_hough_r,
    maxRadius=self.max_hough_r
)

```

עיקרון: כל פיקסל קצה ב **גרדיאנט** מצביע (votes) על מרכזי מעגלים פוטנציאליים במרחב Hough. המעגלים בעלי הקולות הרבים ביותר נבחרים כתוצאה. **היתרון:** עמיד לזיהויים שאינם מעגליים (כמו רגליים).

4.2 זיהוי גמיש -- Contour Analysis

אם Hough כשל והמצב הוא relaxed (כלומר -- המצלמה הצדדית כן ראתה את הכדור), מתבצעת חזרה (fallback) לזיהוי קונטור:

```

contours, _ = cv2.findContours(final_mask, cv2.RETR_EXTERNAL, ...)
if contours:
    largest_contour = max(contours, key=cv2.contourArea)
    area = cv2.contourArea(largest_contour)
    if area > CONFIG["detection"].get("relaxed_min_area", 100):
        ((x, y), radius) = cv2.minEnclosingCircle(largest_contour)

```

4.3 החלקה ותחזית -- Kalman Filter

מסנן קלמן מתוחזק לכל מצלמה. הוא ממלא שני תפקידים:

1. **החלקה** --- הפחתת רעידות במיקום הכדור בין פריימים סמוכים.
2. **תחזית בחסימה** --- במקרה שהכדור נעלם זמנית, המסנן ממשיך לחזות את מיקומו על פי המהירות הקודמת.

```

self.kf = cv2.KalmanFilter(4, 2) # state: [x, y, dx, dy]
self.kf.measurementMatrix = np.array([[1, 0, 0, 0],
                                       [0, 1, 0, 0]], dtype=np.float32)
self.kf.transitionMatrix = np.array([[1, 0, 1, 0],
                                       [0, 1, 0, 1],
                                       [0, 0, 1, 0],
                                       [0, 0, 0, 1]], dtype=np.float32)
self.kf.processNoiseCov = np.eye(4, dtype=np.float32) * 0.03
self.kf.measurementNoiseCov = np.eye(2, dtype=np.float32) * 0.1

```

המודל הוא תנועה לינארית במהירות קבועה (constant-velocity). אם המיקום החזוי יוצא מגבולות הפריים, המסנן מתאפס.

5 זיהוי רצפה באמצעות הומוגרפיה

זהו אחד החידושים החשובים בפרויקט. במקום להסתמך על קו רצפה דמיוני (קו ישר בין שתי anchors), המערכת בונה מודל גיאומטרי מלא של הרצפה.

5.1 הרעיון

עלידי מציאת מטריצת הומוגרפיה (H) שממפה כל פיקסל מהמצלמה למישור הרצפה האמיתי, ניתן לבדוק האם הכדור נמצא בפועל על הרצפה. אם שתי המצלמות מסכימות שהכדור נמצא באותה נקודה במישור הרצפה --- הוא כנראה באמת על הרצפה.

5.2 מתמטית

לכל מצלמה i נחשבת מטריצה $H_i \in \mathbb{R}^{3 \times 3}$ כך ש:

$$\begin{pmatrix} X_w \\ Y_w \\ 1 \end{pmatrix} \sim H_i \cdot \begin{pmatrix} u_i \\ v_i \\ 1 \end{pmatrix}$$

כאשר (u_i, v_i) הם פיקסלי המצלמה ו- (X_w, Y_w) הם הקואורדינטות במישור הרצפה (סנטימטרים). הבדיקה בפועל:

$$\|H_1 p_1 - H_2 p_2\| \leq \varepsilon \implies \text{ball on floor}$$

5.3 מימוש

```

self.H1, _ = cv2.findHomography(pts_cam1, pts_world, cv2.RANSAC)
self.H2, _ = cv2.findHomography(pts_cam2, pts_world, cv2.RANSAC)

def is_ball_on_floor(self, pixel_cam1, pixel_cam2, epsilon=5.0):
    world_p1 = cv2.perspectiveTransform(p1, self.H1)[0][0]
    world_p2 = cv2.perspectiveTransform(p2, self.H2)[0][0]
    distance = np.linalg.norm(world_p1 - world_p2)
    return distance <= epsilon, distance, world_p1, world_p2

```

השימוש ב-RANSAC מאפשר עמידות לטעויות מדידה בודדות בשלב הכיול.

6 שלבי הכיול

המערכת דורשת מספר שלבי כיול עוקבים. כל שלב פותר בעיה אחרת:

6.1 שלב 0: כיול HSV חיצוני

מתבצע פעם אחת באמצעות Calibration Helper script.py. מפעיל GUI עם שישה sliders (H, S, V) עבור החסם התחתון והעליון):

- מציג את הפריים החי ואת המסיכה הבינארית זה לצד זה.
- המשתמש מסובב את ה-sliders עד שהכדור מופיע לבן מלא והרקע שחור מלא.
- התוצאה נשמרת ב-ball_config.json עם פרופיל נפרד לכל מצלמה.

6.2 שלב 1: כיול רקע -- מקש B

איפוס מודל MOG2. השחקן יוצא מהפריים, לוחץ B, והמערכת בונה מחדש את מודל הרקע:

```
def set_instant_background(self, frame):  
    self.fgbg = cv2.createBackgroundSubtractorMOG2(  
        history=CONFIG["background"].get("mog2_history", 500),  
        varThreshold=CONFIG["background"].get("mog2_threshold", 25),  
        detectShadows=False  
    )
```

6.3 שלב 2: כיול רדיוס -- מקש S

המשתמש מניח את הכדור על הרצפה ולוחץ S. המערכת:

1. מבטלת זמנית את MOG2 (זיהוי ב-HSV בלבד --- הכדור עומד במקום).
 2. אוספת 15 דגימות של רדיוס הכדור.
 3. מחשבת את החציון (יציב לחריגים) כרדיוס הבסיסי.
 4. מעדכנת את פרמטרי Hough לטווח של $\pm 50\%$ מרדיוס זה.
- הסיבה לחציון ולא ממוצע: עמידות לזיהויים שגויים בודדים.

6.4 שלב 3: כיול רצפה ב-12 נקודות -- מקש F

זהו הכיול הקריטי ביותר. מתבצע באופן אינטראקטיבי:

1. המערכת מציגה רשת של 12 נקודות (3×4) על הרצפה האמיתית.
2. הקואורדינטות בעולם הן: $X \in \{0, 33, 67, 100\}$ ס"מ, $Y \in \{0, 50, 100\}$ ס"מ.
3. המשתמש מניח את הכדור בכל נקודה, מאמת זיהוי בשתי המצלמות, ולוחץ SPACE.
4. לאחר 12 נקודות, המערכת מחשבת שתי מטריצות הומוגרפיה (אחת לכל מצלמה).
5. התוצאה נשמרת ב-floor_calibration.json.

שים לב: במהלך הכיול נעשה שימוש ב**זיהוי HSV בלבד** (MOG2 מבוטל). הסיבה: הכדור עומד במקום, ולכן מסנן התנועה היה מסלק אותו לחלוטין.

6.5 שלב 4: כיול נגיעת ראש -- מקש H, אופציונלי

השחקן מניח את הכדור על ראשו, לוחץ H, והמערכת מחשבת את הרדיוס המינימלי של הכדור במצלמה העליונה (כאשר הוא קרוב למצלמה). זהו ה-**baseline** לזיהוי נגיעות ראש: כאשר הרדיוס גדל מעבר לסף זה ולאחר ירידה --- זוהי נגיעת ראש.

6.6 שלב 5: כוונן קו רצפה ידני

לאחר הכיולים האוטומטיים, המשתמש יכול לכוון את הרצפה ידנית בזמן ריצה:

- A / Z --- הזזת עוגן שמאלי למעלה או למטה.
- UP / DOWN --- הזזת עוגן ימני (חלופה: K / M).
- [/] --- הזזת קצוות הרצפה במישור X.
- +/- --- כוונן epsilon (סובלנות ההסכמה בין המצלמות) בקפיצות של 0.5 ס"מ.

7 לוגיקת זיהוי בעיטה ונפילה

7.1 זיהוי בעיטה -- Velocity Flip

המערכת עוקבת אחר היסטוריית מהירות הכדור בציר Y (אנכי). **בעיטה** = רצף שבו הכדור היה בנפילה ולפתע מתחיל לעלות:

```
if current_vel > 0:    # falling
    self.fall_counter += 1
    self.rise_counter = 0
    if self.fall_counter >= self.required_inertia_frames:
        self.is_falling = True
elif current_vel <= min_hit_vel:    # rising
    self.rise_counter += 1
    self.fall_counter = 0

# V-Flip: was falling, now rising for 2+ frames
is_normal_hit = (self.rise_counter >= 2) and self.is_falling
```

דרישת inertia_frames מונעת זיהוי שווא של רעידות מצלמה כבעיטה.

7.2 בעיטה ראשונה -- Flick Up

לפני שהכדור באוויר, אין **נפילה** שתקדים את העלייה. ולכן הבעיטה הראשונה דורשת:

1. $\text{rise_counter} \geq 3$

2. מעבר של 15 פיקסלים לפחות כלפי מעלה בשלושה פריימים.

7.3 זיהוי נפילה לרצפה -- שלוש שיטות במקביל

7.3.1 שיטה 1: הומוגרפיה (מועדפת)

אם הכיול ב־12 נקודות הצליח, המערכת בודקת בכל פריים האם שתי המצלמות מסכימות שהכדור על הרצפה (מרחק קטן מ־epsilon):

```
on_ground, distance, __, __ = self.floor_finder.is_ball_on_floor(
    pixel_top, pixel_side, epsilon=self.floor_epsilon_cm
)
if on_ground:
    return "DROP"
```

7.3.2 שיטה 2: קו רצפה ליניארי (fallback)

אם ההומוגרפיה לא כויילה, המערכת משתמשת בקו רצפה דמיוני בין שני עוגנים:

$$\text{floor_y}(x) = \text{floor_left_y} + (\text{floor_right_y} - \text{floor_left_y}) \cdot \frac{x}{w - 1}$$

7.3.3 שיטה 3: ניתוח רטרואקטיבי של תזמון פגיעות

כאשר הכדור קופא במקום למשך 30 פריימים, המערכת מנתחת את היסטוריית הפגיעות האחרונות:

```
for i in range(len(recent_hits) - 1, 0, -1):
    delta_t = recent_hits[i] - recent_hits[i-1]
    if delta_t < 0.35: # too fast for a human kick
        invalid_hits += 1
        floor_chain_detected = True
    elif delta_t < 0.60: # gray zone
        if i > 1:
            delta_t_older = recent_hits[i-1] - recent_hits[i-2]
            if delta_t < delta_t_older - 0.05: # shrinking interval
                invalid_hits += 1
```

עיקרון פיזיקלי: בעיטה אמיתית של אדם יוצרת קצב יחסית קבוע (0.7 ~ שניות), בעוד שכדור הקופץ מהרצפה מאבד אנרגיה והאינטרוולים הולכים ומתקצרים (restitution decay). המערכת מזהה תבנית זו ומחסירה את הקפיצות השגויות מהציון.

7.4 זיהוי נגיעת ראש -- Header

נגיעת ראש מתבצעת באנכי --- הכדור נופל מלמעלה אל הראש ועולה חזרה. במצלמה העליונה, נגיעת ראש מתבטאת בשינוי רדיוס:

1. הכדור נופל ומתרחק מהמצלמה → הרדיוס מצטמצם.

2. הוא מתקרב לראש → הרדיוס גדל.

3. נגיעה → העלייה בעקבות הצטמצמות.

8 מנגנונים נוספים

8.1 טיפול בחסימות -- Lost Frame Handling

המערכת סופרת lost_frames --- פריימים רצופים ללא זיהוי. הזמן המקסימלי מותאם להקשר:

- ברירת מחדל: 30 פריימים.
- אם הכדור היה גבוה ($y < 100$) ובעלייה ($vel < 0$) --- הטיימאאוט מוארך ל-90 פריימים (הכדור בוודאי באוויר מעל המסגרת).

8.2 שער רדיוס -- Radius Gate

זיהויים שהרדיוס שלהם חורג מהטווח $[0.5R_b, 1.5R_b]$ נדחים אוטומטית. זה מסנן זיהויים שגויים של אובייקטים אחרים (פנים, ידיים).

8.3 משחק רב משתתפים

- מקש T --- ספירה לאחור של 3 שניות ותחילת המשחק.
- מקש N --- מעבר בין שחקנים, שמירת תוצאה.
- לאחר ששני השחקנים שיחקו --- מוצג מסך מנצח.
- התוצאות נשמרות ב-`evaluation_log.csv` עם אחוז דיוק (Clean Play %).

8.4 משוב קולי

בעת זיהוי נפילה, המערכת מנגנת קובץ `referee-whistle-sound-effect.mp3` ב-`pygame` בתהליכון נפרד כדי לא לחסום את הלולאה הראשית.

9 תצוגה וויזואליזציה

9.1 Floor Overlay

- המערכת מציירת על שני הפריימים את רשת ה-12 נקודות שכולה:
- ארבעת הפינות יוצרות מצולע (polygon) מלא שקוף.
 - הקווים האופקיים והאנכיים מצוירים בין הנקודות.
 - הצבע משתנה לאדום למשך חצי שנייה לאחר זיהוי נפילה --- משוב ויזואלי מיידי.

9.2 הודעות תוצאה

- +1 מופיע במרכז המסך לאחר כל בעיטה ועולה למעלה בהדרגה תוך שנייה אחת (fade-out).
- תיבת P1: X | P2: Y בפינה הימנית העליונה מציגה את ציוני שני השחקנים.
- DROP! GAME OVER מוצג במרכז המסך בצבע אדום.

10 קבצי קונפיגורציה

- **config.py** --- קונפיגורציה סטטית (פרמטרים שלא משתנים בריצה).
- **ball_config.json** --- גבולות HSV לכל מצלמה (נשמרים בכלי הכיול).
- **floor_calibration.json** --- 12 נקודות הכיול במישור העולם ובשתי המצלמות.
- **runtime_config.json** --- פרמטרים שמשתנים בזמן ריצה (כגון floor_epsilon_cm).
- **evaluation_log.csv** --- לוג תוצאות משחקים: ציון, דדאקציות, משך, אחוז דיוק.

11 הערכה -- Evaluation

הפרויקט כולל מערכת הערכה אובייקטיבית:

- **capture_eval_frames.py** --- לכידת פריימים לסט בדיקה.
- **generate_ground_truth.py** --- יצירת תיוג ידני לפריימים.
- **evaluate_segmentation.py** --- חישוב מדדים (IoU, Precision, Recall) של איכות הסגמנטציה מול ה-ground truth.
- **run_pipeline.py** --- הרצת הצינור על סט הבדיקה.

12 סיכום ומסקנות

הפרויקט משלב מספר טכניקות ראייה ממוחשבת קלאסיות לפתרון בעיה מעשית, תוך התמודדות עם אתגרים אמיתיים של זיהוי אובייקט מהיר:

1. **שילוב מרובה שיטות** --- HSV + MOG2 + Hough + Kalman + Homography --- כל שיטה משלימה את החולשות של האחרות.
2. **שתי מצלמות** --- מאפשרות הסכמה גיאומטרית ומפחיתות False Positives.
3. **כיולים מדורגים** --- כל שלב פותר בעיה ספציפית, וניתן לדלג על שלבים אופציונליים.
4. **ניתוח רטרואקטיבי** --- שימוש בפיזיקה של קפיצות-רצפה לזיהוי בדיעבד של נפילות לא מזוהות.
5. **ארכיטקטורה מודולרית** --- הפרדה ברורה בין רכישה, עיבוד, לוגיקה והצגה.

המערכת הוכיחה את עצמה ככלי שימושי לאימון ז'אנגלינג ולתחרויות בלתי פורמליות, עם משוב בזמן אמת ותחזוקה נמוכה לאחר הכיול הראשוני.

מסמך זה נוצר אוטומטית מקוד המקור של הפרויקט.